

I have a requirement to make an HRMS and vendor management kind of system. The situation is like this: one PSU company hires vendor employees through NICSI, and they want a way to manage purchase orders, Work orders, Advance payments, vendor employee details, and attendance.

It happens like this: Suppose there are multiple projects in the company and there is a requirement for more manpower, so the project manager can raise a purchase order request through SAP, and the purchase department will handle the purchase order process; we only need to record and track the Purchase order number(PO No.) and other basic

Show more ▾

This is actually a very good enterprise-level project. It is closer to an ERP module than a simple HRMS because it combines **Procurement + Vendor Management + HR + Attendance + Finance + Project Management**.

From your explanation, I would call the system something like:

Vendor Resource & Advance Management System (VRAMS)

or

Vendor Workforce Management System (VWMS)---

1. First understand the business flow

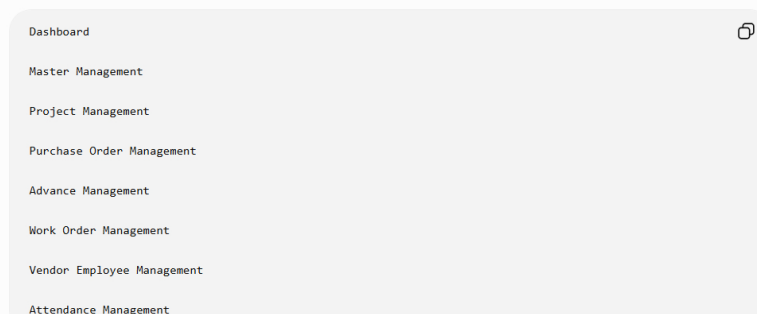
The entire process can be represented as



So the cycle repeats forever.

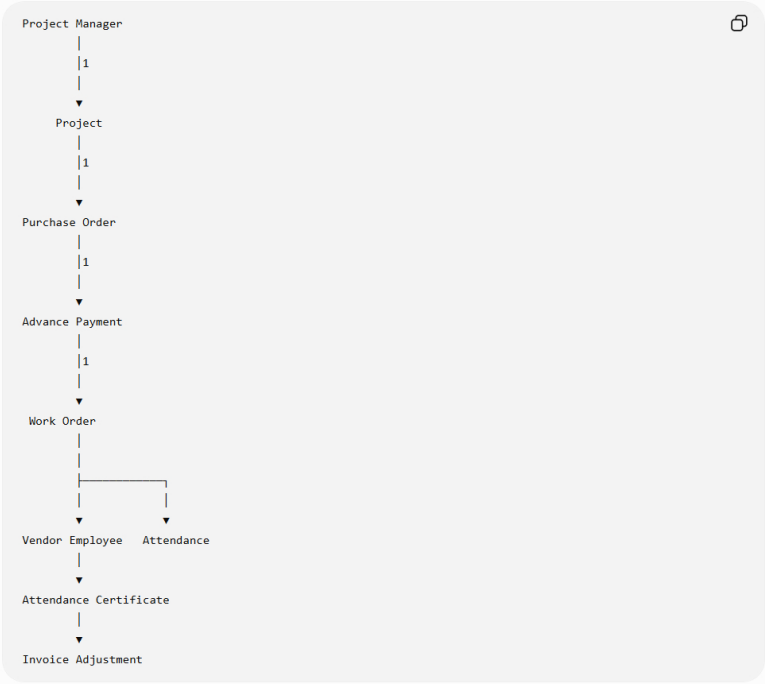
2. Main Modules

I would divide the application into eight modules.



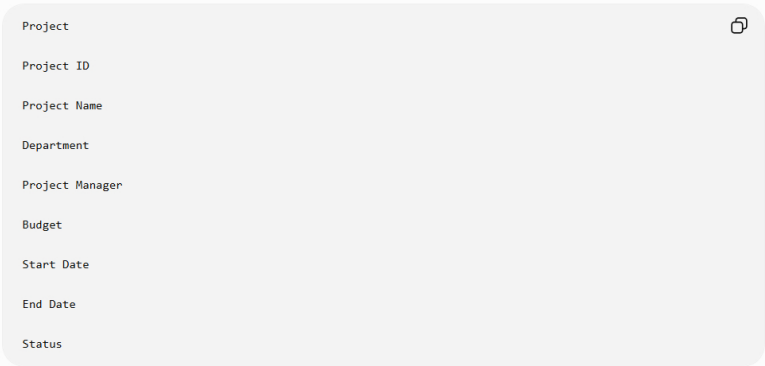
3. Complete Entity Relationship

Instead of thinking in tables, think in business entities.

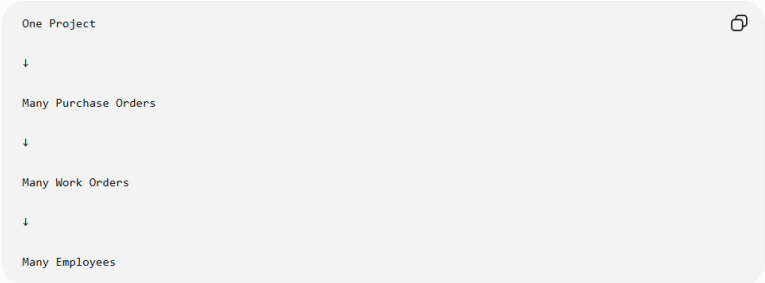


4. Project Entity

This is the PSU's internal project.

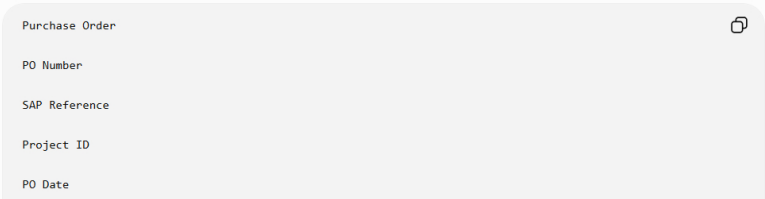


Relationship



5. Purchase Order Entity

This is only for tracking because SAP already creates it.



Vendor
Required Resources
Expected Cost
Status
Remarks

Status

Requested
Approved
Advance Pending
Advance Paid
WO Generated
Completed

One PO

↓

One Advance

↓

One WO

Exactly as your requirement says.

6. Advance Payment

This is the finance module.

Advance
Advance ID
PO Number
WO Number
Project
Amount
TDS
GST
Net Amount
Bank Reference Number
Cheque Number
Payment Date
Payment Mode
Remaining Balance
Status

Example

Advance
12,00,000
TDS
1,20,000
Paid
10,80,000

Then every month

Salary Adjustment
1,90,000
Advance Left
10,10,000

Eventually

Advance Left

1,00,000

Dashboard should always show

Total Advance Paid

Advance Utilized

Advance Remaining

7. Work Order

Work Order comes from NICSI.

WO Number

PO Number

Project ID

NICSI Project ID

WO Date

Start Date

End Date

Total Resources

WO Amount

Status

Notice

NICSI Project ID

↓

One only

PSU Project IDs

↓

Many

So save both.

8. Vendor Employee Master

This is your HR module.

Vendor Employee

Employee Code

Name

Father Name

DOB

Gender

Mobile

Email

Address

PAN

Aadhaar

PF Number

ESIC Number

Insurance

Qualification

Experience

Joining Date

Relieving Date

Agency
NICSI Reference
CV
Photo
Documents

One employee belongs to

One Work Order	
One Project	
One Vendor	

9. Attendance

Daily attendance

Attendance	
Employee	
Date	
In Time	
Out Time	
Present	
Half Day	
Leave	
Absent	
Remarks	

Monthly

Attendance Certificate	
Certificate Number	
Project	
Month	
Year	
Generated Date	
Approved By	
PDF	

10. Monthly Invoice Adjustment

This is extremely important.

Instead of modifying advance directly, create transactions.

Advance Ledger	
Entry ID	
Advance ID	
Date	
Type	
Amount	
Reference	
Balance	

Types

Advance Paid	
Salary Adjustment	
Refund	
Additional Advance	

Example

12,00,000 Paid



↓

Balance

12,00,000

Salary Invoice

2,00,000

↓

Balance

10,00,000

Salary Invoice

2,00,000

↓

Balance

8,00,000

...

This is exactly how accounting systems work.

Never update balances manually.

Always calculate from ledger.

11. Project Dashboard

Every Project Manager should see

Project



↓

Total PO

↓

Total WO

↓

Employees Working

↓

Attendance %

↓

Advance Paid

↓

Advance Remaining

↓

WO Expiry

↓

Pending Attendance Certificate

12. Admin Dashboard

Overall PSU

Total Projects



Total Project Managers

Total Vendor Employees

Total Active Employees

Total Advance

Advance Remaining

Expired WOs

Expiring WOs

Employees without Attendance

Pending Certificates

Pending Advances

13. Workflow

Admin Creates Project

↓

Assign Project Manager

↓

Project Manager enters PO

↓

Advance Payment

↓

Finance enters Bank Ref

↓

WO received

↓

WO uploaded

↓

Employees added

↓

Attendance starts

↓

Monthly Attendance Certificate

↓

Invoice Adjustment

↓

Advance Reduced

↓

WO Expired

↓

Need manpower?

↓

New PO

↓

New Advance

↓

New WO

↓

Continue

14. Folder Structure (Spring Boot)

src

controller

service

repository

entity

dto
mapper
config
security
exception
util
scheduler
report
attendance
purchaseorder
workorder
advance
project
vendor
dashboard

15. React Structure

src
pages
Dashboard
Projects
Purchase Orders
Work Orders
Advance
Attendance
Vendor Employees
Reports
Masters
components
tables
forms
cards
charts
layout
services
api
hooks
utils

16. Database Design

Users
ProjectManagers
Projects
PurchaseOrders
Advances
AdvanceLedger
WorkOrders
VendorEmployees
Attendance

AttendanceCertificate
Invoices
Documents
Notifications
AuditLogs

17. Suggested Technology Stack

Since this is an enterprise-style application, I recommend:

Backend

- Java 21
- Spring Boot
- Spring Security (JWT)
- Spring Data JPA
- Hibernate
- Validation
- Lombok
- MapStruct
- PostgreSQL (or SQL Server if aligning with enterprise infrastructure)
- Flyway for database migrations

Frontend

- React
- TypeScript
- Vite
- Material UI
- React Query (TanStack Query)
- React Hook Form
- Zod
- Recharts (dashboards)

Storage & Reporting

- MinIO or cloud/object storage for CVs and documents
- JasperReports or PDF generation library for attendance certificates

DevOps

- Docker
- Docker Compose
- GitHub Actions (CI/CD)
- Nginx (deployment)

Key Design Recommendation

One of the most important design decisions is to **treat the advance as a ledger instead of a single balance field**. Every advance payment, invoice adjustment, refund, or correction should create a transaction entry. The current balance is derived from the transaction history, providing a complete audit trail, easier reconciliation with finance records, and reliable reporting.

Overall, this is a solid **enterprise ERP module** that combines procurement tracking, vendor workforce management, financial advance reconciliation, attendance, and project management. If implemented with clean domain separation and a ledger-based financial model, it would closely resemble the architecture used in commercial ERP systems while remaining tailored to your PSU–NICSI workflow.

📄 🖨️ 🔍 ⬆️ ↺️ ⋮ | 🗑️

What exactly is a ledger-based financial model here for the advance? Also suggest other improvements. And the admin will only create the Project manager profiles and the project managers will handle the projects under them, the admin is mainly for monitoring.

Your clarification actually makes the system design much cleaner. If the **admin is only for governance and monitoring**, then this becomes a decentralized workflow where **Project Managers own the operational data**, while the **Admin has organization-wide visibility**.

The first thing I'd change is the architecture slightly.

Admin	📄
├─ Create Project Managers	
├─ View All Projects	
├─ View All Advances	
├─ View Attendance Reports	
├─ Generate Organization Reports	
└─ Manage Masters	

```
└─ Audit Logs

Project Manager
|
├─ Create Project
├─ Add PO
├─ Record Advance
├─ Upload WO
├─ Add Vendor Employees
├─ Mark Attendance
├─ Generate Attendance Certificate
├─ Record Invoice Adjustment
└─ Close Project/WO
```

So the admin almost never enters operational data.

Now, about the Ledger Model

Most people make this mistake:

They create

```
Advance

Amount Paid = 12,00,000

Remaining = 9,75,000
```

Every month they update Remaining.

Month 1

```
Remaining = 10,00,000
```

Month 2

```
Remaining = 8,00,000
```

Month 3

```
Remaining = 6,25,000
```

After two years nobody knows

- why it became 6.25 lakh
- who changed it
- whether someone made a mistake
- whether an invoice was reversed

So accounting systems **never** work like this.

Instead they maintain a transaction history.

Imagine this table.

Date	Transaction	Credit	Debit	Balance
1 Jan	Advance Paid	12,00,000		12,00,000
31 Jan	Invoice Jan		2,00,000	10,00,000
28 Feb	Invoice Feb		2,00,000	8,00,000
31 Mar	Invoice Mar		1,90,000	6,10,000
30 Apr	Invoice Apr		2,00,000	4,10,000

Nothing gets deleted.

Nothing gets overwritten.

Every movement is recorded.

This table is called a **ledger**.

Suppose in May NICS refunds ₹50,000.

Instead of editing balance

just add

Date	Transaction	Credit	Debit
5 May	Refund	50,000	

Balance becomes

4,60,000



Automatically.

Suppose finance entered the wrong invoice.

Instead of deleting

Add

Reverse Invoice



Credit

2,00,000

then

Correct Invoice



Debit

1,75,000

Everything remains auditable.

In your project

I would have

Advance



Advance ID

PO

WO

Project

Initial Amount

and another table

Advance Transactions



ID

AdvanceID

Transaction Type

Amount

Reference Number

Transaction Date

Remarks

Created By

Transaction Type

ADVANCE_PAYMENT



MONTHLY_INVOICE

REFUND

ADDITIONAL_ADVANCE

TDS

GST

ADJUSTMENT

REVERSAL

Then

Current Balance

=

Total Credits



-

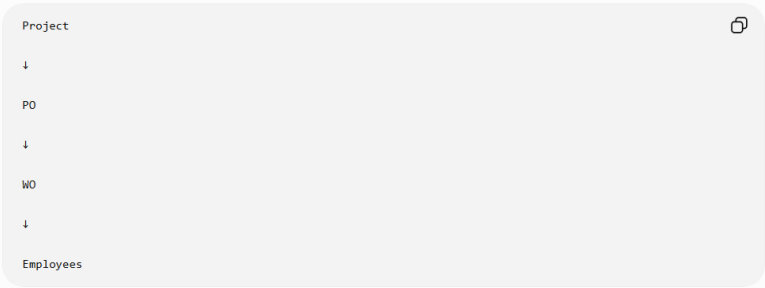
Total Debits

No "Remaining Amount" column is actually needed.

Even Better

I would actually introduce a **Vendor Contract** entity.

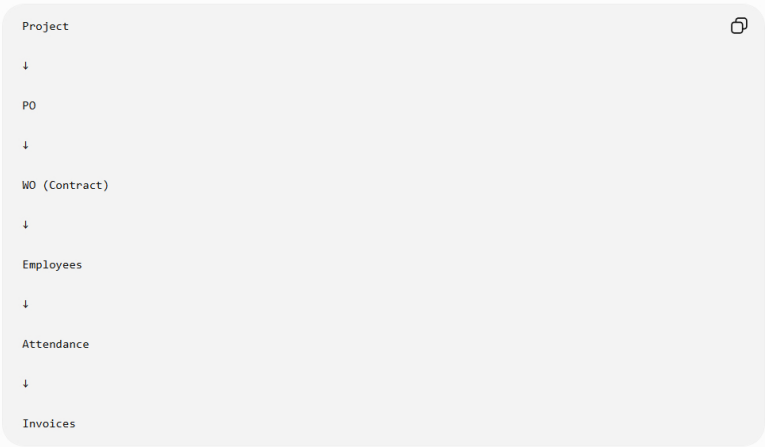
Right now you have



But in reality

the Work Order itself is the agreement.

So



Everything revolves around the WO.

Another Improvement: Employee History

Employees can return after six months.

Instead of creating them again

have

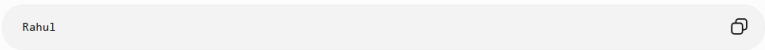


and



Example

Employee



Deployments

Project	WO	From	To
Project A	WO101	Jan-Jun	
Project C	WO220	Jul-Dec	

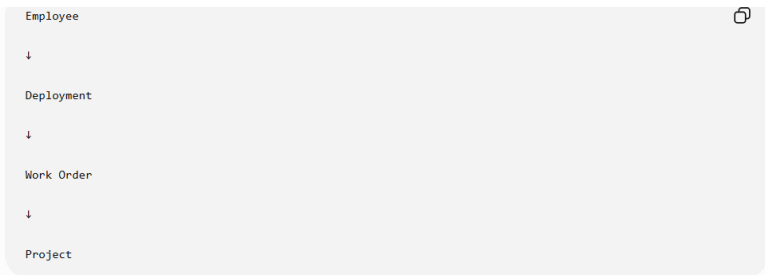
Same employee.

Different deployments.

This preserves employment history and avoids duplicate employee records.

Don't Attach Employees Directly to Projects

Instead



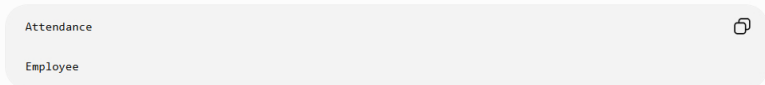
because if an employee changes projects

nothing changes in Employee Master.

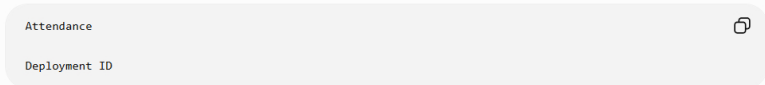
Only Deployment changes.

Attendance Should Also Be Linked to Deployment

Instead of



use



Otherwise

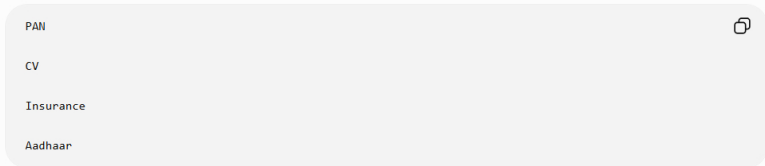
If Rahul joins Project B

his attendance history becomes confusing.

Deployment solves this.

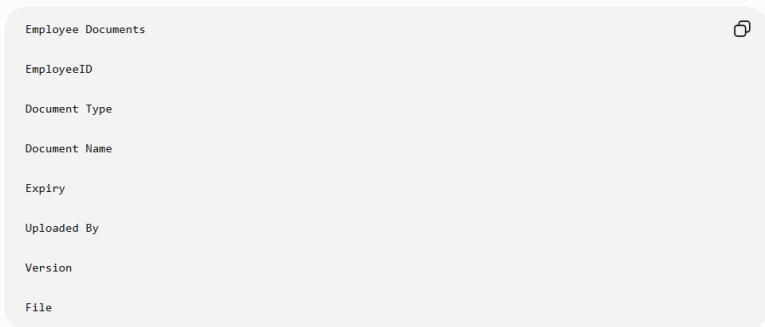
Document Management

Instead of storing



inside Employee

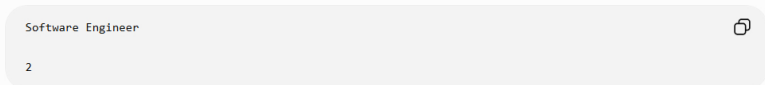
make



Then adding new document types later requires no schema changes.

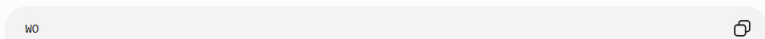
Work Order Resource Planning

Suppose WO says



But only one joined.

Track that.



Required Resources = 2

Joined = 1

Vacant = 1

This helps PMs immediately identify staffing gaps.

Advance Dashboard

Instead of just

Advance Left



show

Initial Advance



Utilized

Remaining

Utilization %

Burn Rate

Estimated Exhaustion Date

Example

Initial



12 lakh

Utilized

11 lakh

Remaining

1 lakh

Average Monthly Burn

2 lakh

Advance lasts

15 days

This becomes extremely useful.

Notifications

This system should notify users when:

- WO expires in 30/15/7 days
- Advance balance falls below a threshold (e.g., 10%)
- Required resources are not fully onboarded
- Monthly attendance certificate hasn't been generated
- Employee documents (insurance, ID proofs, etc.) are nearing expiry
- A project has no active WO but still has deployed employees

Suggested Overall Architecture

Rather than thinking of it as an "HRMS," model it around five core domains:

Identity Module
(Admin, Project Managers)



↓

Project Module

↓

Procurement Module
(PO, Advance, WO)

↓

Vendor Workforce Module
(Employee Master, Deployment)

↓

Operations Module
(Attendance, Certificates, Invoices)

↓

Finance Module
(Advance Ledger, Utilization, Reports)

↓

Dashboards & Analytics

This domain-driven structure will scale much better as new requirements emerge (multiple vendors, additional payment types, approval workflows, etc.) than a traditional CRUD-based design.

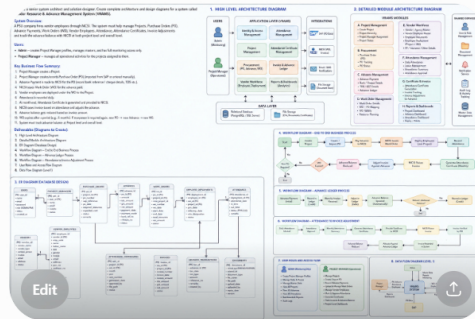
One more improvement I would strongly recommend

Right now, you mentioned **one PO has one WO**. Even if that's the current business rule, I would **design the database to allow one PO to have multiple WOs** (a one-to-many relationship) while enforcing the current one-to-one rule in the application.

Why? Business rules change. If, in the future, NICS! splits a PO into two work orders (for example, different resource batches or extensions), your database won't need a redesign—only a change in the application logic. Designing for this flexibility now costs almost nothing and can save significant effort later.



Give me a detailed prompt to give to erasor.io to make the complete architecture diagram. And also the ER diagram and the workflow diagrams.



Edit Image



+ Ask anything



ChatGPT can make mistakes. Check important info.